

RETRACE – Game Design & Implementation Blueprint

Executive Summary: RETRACE is a 2D top-down sci-fi stealth/puzzle game. The player is **Subject 047**, trapped in a futuristic research facility. The core loop combines **ECHOs** (time-clone echoes of past attempts), a learning enemy **MIMIC**, limited sight and sound stealth, and puzzle-solving. Key mechanics: the player can RETRACE (restart) up to 3 times, creating up to three active ECHO clones that exactly repeat previous runs; MIMIC observes and learns the player's actions (unlocking doors, sprinting, etc.) to adapt its behavior [58†L221-L229] ; player vision is confined by walls/doors (true line-of-sight); sounds (footsteps, doors) reveal threats; and a **hidden 3-life** "Temporal Stability" system triggers progressive screen glitching on each capture. The goal is to restore power, bypass security, disable containment, and reach the surface while outsmarting the self-learning MIMIC. This document details **all game systems**, with design rationale, technical specs, diagrams, puzzle examples, reference images, and an implementation roadmap with Claude Code prompt outlines.

Game Overview

Retrace is set in a **dimly-lit sci-fi facility** with distinct wings (Research, Security, Temporal Labs, etc.). The player awakens with no memory and must **escape**. The facility's AI (MIMIC) believes the player should *not* leave, so it actively hunts the player. Gameplay alternates between planning (choosing routes and using ECHOs as distractions/helpers) and execution (stealth movement with limited vision).

- **Objective Structure:** The ultimate exit door is locked by three systems (e.g., **Power, Security Authorization, Containment Override**). Each requires exploration of different wings and solving multi-step puzzles using ECHOs and stealth. Only by completing all three objectives does the exit unlock.
- **Narrative Hints:** Terminals and logs reveal the backstory (Project RETRACE, Subject 047) and explain the hidden life count ("temporal stability") **after** the player experiences it. This environmental storytelling rewards exploration.

Core Mechanics – ECHOs

The **ECHO** system is the heart of RETRACE. An ECHO is a replay of a previous attempt. During play, the game continuously records the player's movement and actions. When the player **RETRACES** (presses and holds **R**), the current attempt is saved as an ECHO and the player resets to the last checkpoint. Up to **3 ECHOs** can exist at once. On playback, all ECHOs **simultaneously replay** their recorded actions in sync with the player's current run.

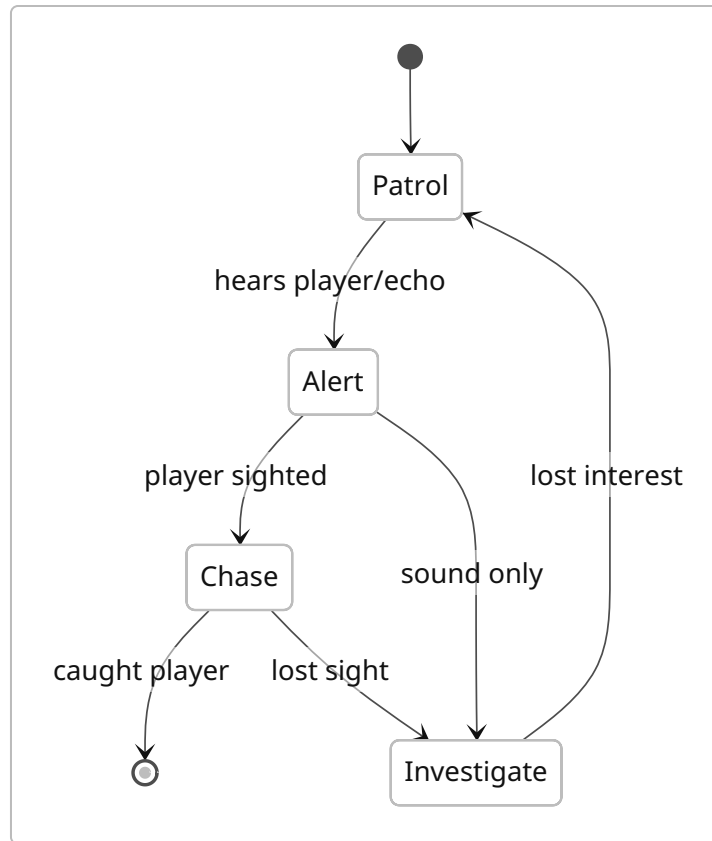
- **Recording Format:** Each ECHO stores a timestamped sequence of player states (position, velocity, and action flags like "pressed button", "opened door", etc.) at a fixed rate (e.g. 60 Hz). Colliders or triggers activated by the player are reactivated by ECHOs at the recorded times.

- **Playback & Synchronization:** On each frame of a RETRACE run, for each ECHO we set its position and state according to the recording at that time index. All ECHOs move in lockstep with the player's current time. Care is taken so physics-dependent actions (e.g. holding a pressure plate) match perfectly with recording (tile-based indexing or deterministic physics).
- **Replacement Policy:** When the player RETRACES and already 3 ECHOs exist, the **oldest** ECHO is discarded, the remaining shift (Echo #2→#1, #3→#2), and the new run becomes the new Echo #3. This encourages strategic use: losing an extra life won't create an ECHO, so RETRACING intentionally (R key) is encouraged to manage Echo lineup.
- **Multi-ECHO Coordination:** Puzzles are designed so that **multiple ECHOs are required**. For example, one Echo might hold a switch while another opens a gate elsewhere. This is mandatory, not optional: every key puzzle uses ≥ 2 ECHOs. It forces creative time-planning. *Citing example:* games like *Time Clones* use a similar mechanic – “every clone you produce is a time-traveling double that follows every step you've taken before” [50†L32-L40] .

Echo System	Requirement	Implication
Max Active ECHOs	3 (plus current player = 4 agents)	Limits puzzle complexity and performance.
Recording Resolution	~60 samples/sec or frame-locked	Balances smooth replay vs. memory cost.
Input vs Physics-based	Record actions & positions, not low-level physics	Ensures determinism of movements.
ECHO Sound Replay	Echos replicate original sounds	Reinforces stealth tension.

MIMIC AI

MIMIC is an autonomous enemy robot with a single large eye. It has a **state machine** and a learning component:



- **States:** MIMIC starts **Patrolling** (wandering assigned routes). If it **hears** a footstep or door (above a threshold), it goes to **Alert** and moves toward the sound. If it **sees** the player or an ECHO within its vision cone (at line-of-sight, see below), it enters **Chase**. If it loses sight of the target, it goes to **Investigate** (searching around last-known location), then returns to Patrol. When in Chase, if it physically contacts the player, it catches them and triggers a RETRACE.
- **Vision Cone:** MIMIC has a forward cone (e.g. 90° FOV, range ~5 tiles). It cannot see through walls or closed doors. We compute this via raycasts (see Visibility System).
- **Hearing:** Footsteps or doors play directional audio (see Sound System). If the player or an Echo makes enough noise (within ~10 tiles), MIMIC turns toward it and investigates.
- **Learning Behavior:** Crucially, MIMIC **remembers** the player's behavior across attempts. It logs what paths the player took, where they hid, and what actions (running, shooting, opening specific doors) occurred. Over multiple trials, MIMIC statistically favors areas and moves used by the player. For instance, if the player always uses the east corridor, MIMIC will start patrolling it. This is a simple rule-based adaptation (e.g. increment a counter for each segment or action used; more-used actions become MIMIC capabilities). *As seen in the game ECHO: "whenever [En] performs an action... the clones... learn to use it against her. Each reboot updates them, learning En's actions during that cycle" [58†L221-L229]* . Similarly, MIMIC will copy actions: if the player sprinted or opened doors, MIMIC will do so after learning it. Unlike a full neural AI (complex, data-hungry), we use a procedural adaptation model (pros/cons in table below).

Learning Approach	Description	Pros	Cons
Rule-based (our choice)	Increment counters on player actions/paths	Predictable design; low CPU usage [58†L221-L229]	Less “surprising” intelligence
Machine Learning	Train on player traces (e.g. neural network)	Potentially more nuanced, less scripted	High complexity; overkill for small scope

- **MIMIC Memory Across Retries:** On a full timeout (3 catches → reset to start), all **Level/Objective progress remains** (visited rooms, solved terminals), but most learned data resets to a fraction (e.g. half counters) to avoid unbeatable behaviors. MIMIC **does not forget completely**, setting up a meta-plot question (why is it aware across resets?).

Visibility & Lighting

RETRACE uses a true **2D line-of-sight (LOS)** system. The player only sees what their character could physically see on the map.

- **Raycast LOS:** Every frame, we cast rays from the player’s position in multiple directions (e.g. for each tile edge) until hitting a wall, closed door, or large obstacle. Only tiles hit by an unobstructed ray are “visible” (full brightness). This creates non-circular vision shadows. *As one game-dev guide explains, visibility can be treated as “walls and shadows in 2D” and rendered using stencil or shadowcasting techniques [\[53†L139-L148\]](#)* . We prefer continuous raycasts for accuracy, but see table below for alternatives.
- **Blocking Rules:**
 - **Walls & Closed Doors:** Opaque, completely block vision and cannot be walked through.
 - **Open Doors / Windows:** If open, cast rays through; if glass, allow visibility but not passage.
 - **Large Objects:** Some furniture or barriers (e.g. pillars) can block partially.
 - **Visibility States:** Each tile has three states: **Visible (current LOS)**, **Explored (seen before, now in darkness)**, **Unseen**. Explored tiles are drawn dark grey (outline or minimal detail) – the player remembers the layout but not current activity. Unseen tiles are black. This is essentially a fog-of-war. Crucially, if MIMIC or an Echo moves into an explored-but-not-visible area, the player **does not see them** (adding to suspense).
 - **Lighting Effects:** Lights in rooms create *pools* of illumination (e.g. small bright circles) to accent the atmosphere, but do *not* override LOS blocking. Flickering lights (during power loss) reduce LOS range.

Visibility Method	Description	Pros	Cons
Grid shadowcasting	Precompute FOV per tile (tile-based LOS)	Fast on tile grid; simple logic	Coarse; not as smooth as continuous rays
Continuous raycast	Cast many rays (per-degree or per-edge) each frame [53†L139-L148]	Accurate, smooth shadows	More CPU for many rays per frame

By default, we target **60 FPS**. LOS raycasts are limited to a short radius (e.g. 6 tiles) and optimized (e.g. use a field-of-view polygon per frame) to stay performant.

Sound System

Sound augments LOS to create tension. Key points:

- **Footsteps and Doors:** Each step or door/opening generates a sound event at the tile. If MIMIC is within hearing range (e.g. radius ~10 tiles) and the sound is above a threshold, an indicator arrow briefly appears at the edge of the screen pointing toward the sound (so the player can guess source). MIMIC will go investigate that location (transition to Alert state).
- **Echo Sounds:** When Echo clones walk or act, they also reproduce the original sounds. This means triggering an Echo can deliberately lure MIMIC (or distract it) by playing recorded footsteps.
- **Environment Sounds:** Cameras (hum), generators, alarms etc. play ambient loops. Hearing an alarm or unusual machine sound might also attract MIMIC or signal danger.

Temporal Stability (Lives) and Glitch Effects

RETRACE hides the “3 lives” as a narrative mechanic. Each time MIMIC captures you, you **lose one hidden Temporal Stability point** and the game visually “glitches”.

```
flowchart LR
    3/3 --> 2/3
    2/3 --> 1/3
    1/3 --> 0/3
    2/3 -->|glitch| play continues
    1/3 -->|severe glitch| play continues
    0/3 -->|collapse| Full reset to start
```

- **Glitch Effects:** On first catch (2/3 remaining), introduce minor screen artifacts: brief flicker, slight color distortion, faint static on UI. On second catch (1/3), increase effect intensity: UI text occasionally garbles, lights short-out briefly, ECHOs flicker. On final catch (0/3), **full blackout** occurs: the screen rapidly fills with glitch noise (as shown in the figure below) before cutting to black and resetting. The glitch imagery is inspired by digital static and color channel shifts:

【44tembed_image】 *Figure:* Example of a **glitch overlay** visual effect (abstract static/artifacts). Use this style of distortion on the UI and screen to represent timeline instability.

【47tembed_image】 *Figure:* Another **multicolored glitch/noise** pattern. Intensify these distortions on 2nd and 3rd catch.

- **Reset Behavior:** Upon stability loss (0/3), we display a brief message (“**Timeline Failure – Restoring Baseline...**”) and then reset the player to the last major checkpoint (e.g. start of wing or last unlocked door). All puzzle states that weren’t permanently solved revert (switches, doors), but permanent progress remains: discovered map areas stay marked, unlocked sectors stay open, and story log entries persist. MIMIC retains a portion of its learned data (e.g. half the counters) so each run builds on knowledge.

This 3-strikes rule adds tension: the player never sees a “lives” counter, only the in-world glitch feedback. Only after the final collapse would the player figure out the mechanic (later confirmed via an in-game log).

Level & Puzzle Design

The facility is divided into thematic sections (e.g. **Power Wing, Security Wing, Temporal Labs, Control Center**). The progression is mostly linear (power → security → containment), but with backtracking through already-seen corridors that now can be used differently after solving puzzles. Each area contains multi-step objectives that require ECHOs and stealth to solve. We recommend a modular **room-and-corridor** approach (prebuilt room templates with variations). Common room types: security checkpoint, computer terminal room, generator chamber, office, lab with glass partitions, maintenance ducts.

Multi-ECHO Puzzle Examples

Below are six example puzzles. In each, numbered steps outline required actions by Player and Echo clones.

1. **Double Switch Doors:** Two separated electrical switches (A and B) must be ON simultaneously to open a heavy gate.

2. *Setup:* Switch A on left side, Switch B on right, gate in middle. MIMIC patrols centrally.

3. *Solution:*

1. First, play and hit RETRACE early to create Echo #1.
2. In second attempt (Echo #1 running path 1): Echo #1 goes to Switch A and holds it down.
3. Player (current) runs to Switch B and holds it. Now both A and B are pressed (Echo #1 + Player), gate opens.
4. Player proceeds through open gate while Echo #1 stays on A.

4. *Echo Roles:* Echo #1 = holds Switch A; Player = holds B. (If Echo #2 existed, it could distract MIMIC, etc.)

5. **Parallel Button Rounds:** Three buttons (X,Y,Z) each send a timed pulse to a device; all three pulses must align. One button is behind a locked door.

6. *Setup:* Buttons X and Y are immediate; Button Z is behind Door which requires a key. MIMIC guards X/Y area.

7. *Solution:*

1. Run and collect Key for Z, then press X; get caught by MIMIC while trying to reach Y. (Echo #1: Key+X)
2. Echo #1 (holding Key and pushing X). Player now in Y area (new run) presses Y and goes through unlocked Door (opened via Key).
3. Now all three signals X,Y,Z have been triggered (by Echos), opening the device.

8. *Echo Roles*: Echo #1 = collected key & pressed X; Echo #2 (if used) could hold Y while Player triggers Z, etc.
9. **Camera Node Bypass**: A security camera observes a hallway. Beyond it is a locked vault. A console in another room toggles the camera.
10. *Setup*: Hallway with camera (visible by player), door to vault locked. Nearby room has a camera control console, but operating it triggers an alarm sound.
11. *Solution*:
1. Create Echo #1. Let Echo #1 walk into camera's view (triggering MIMIC) and stay. Player (in next run) quietly sneaks to control room.
 2. Player presses console (alarming MIMIC to Echo's position). Meanwhile, Echo #1 has drawn MIMIC away.
 3. Player safely accesses vault (camera now disabled by console action).
12. *Echo Roles*: Echo #1 = sacrifices itself as bait on camera.
13. **Laser Grid with Reflector**: A corridor has a laser beam barrier that activates on weight plate. Across the beam is a mirror block.
14. *Setup*: Pressure plate A on one side triggers a lethal laser wall in the corridor. A mirror tile sits on the other side of beam. The corridor beyond is required to reach an objective.
15. *Solution*:
1. Echo #1 holds down Plate A, keeping the laser on. Player uses this run to push the mirror into the beam (deactivating it by reflecting it away from trap).
 2. Now laser is disabled (mirror held correctly) and with Echo #1 still on plate, Player can safely run through and fetch goal.
16. *Echo Roles*: Echo #1 = holds Plate A (keeps trap active); Player = moves mirror to disable trap.
17. **Staggered Key and Race**: Two identical keys (K) and one locked exit door. However, MIMIC knows which key was used. The key must be on pressure switch for a second to open door.
18. *Setup*: Keys K1 and K2 in separate rooms. The exit door requires the *unused* key to open (player wakes with code "don't use K1").
19. *Solution*:
1. Play and use K1 on Door (fail and get caught). (Echo #1 now "knows" K1 was used.)
 2. Echo #1 holds K1 on pressure for 2s to no avail. Player goes to get K2 and uses it on Door. The system unlocks because it's different.
20. *Echo Roles*: Echo #1 = used wrong key once (accidentally "teaches" MIMIC/Keeper not to trust K1).

21. **Multi-Pressure Puzzle:** A room has four pressure plates; a heavy crate blocks plate D. Two plates must be down to open the final door, but stepping on three plates triggers an alarm.
22. *Setup:* Plates A, B, C, D. Crate covers D. Need exactly A and C (for example). MIMIC patrols outside room.
23. *Solution:*
1. In Run1, push Crate onto D and hold B (create Echo #1).
 2. In Run2, Echo #1 stays on B, Player jumps to press A and steps out immediately (triggers nothing lethal).
 3. In Run3, Echo #1 still on B, Player steps on C and runs out. Now A + C have been stepped (not B or D) which is the correct combination → door opens.
24. *Echo Roles:* Echo #1 = holds B; Player triggers A and C in separate runs.

These examples illustrate the **simultaneity and coordination** of up to 3 ECHOs plus the player. Designing puzzles around ECHOs gives RETRACE its identity. (For reference, games like *Time Clones* encourage creative use of clones for buttons and platforms [50†L32-L40] .)

UI/UX

The UI must be **minimal** to maintain immersion. Key elements:

- **HUD:** No visible “lives” or heart icons. Instead, use context cues: e.g. a small corner text “Stability: ___” that glitches/disappears as lives drop (eventually hidden). Show **active ECHO count** (e.g. icons ●○○) to indicate how many echoes are currently available.
- **Actions:** A simple prompt “[E] Open/Interact” appears contextually. Pressing **R** shows a small “RETRACE” meter (progress circle) while held.
- **Sound Indicators:** When MIMIC is off-screen but aware of player, a subtle alert icon or sound wave appears at screen edge. Directional arrows for noises.
- **Camera Terminals:** Hacking a security terminal brings up a split-screen showing live camera feeds. Use a simple UI with camera numbers. (See figure below for minimalist UI inspirations.)

【26†embed_image】 *Figure:* Example dashboard/HUD design. Notice the clean overlays and minimal text — suitable for an in-game terminal or player stats.

【27†embed_image】 *Figure:* Stylized UI layout sketches. A simple blocky font and wireframe designs can fit our geometric aesthetic for menus.

【28†embed_image】 *Figure:* UI wireframe sketches. Embrace minimalist layouts for UI elements (bottom toolbar, screens).

【29†embed_image】 *Figure:* A modern interface (graphs and text). We will not be this complex, but this shows how to cleanly present data (e.g. a security log).

【31†embed_image】 *Figure:* An analog gauge. This inspires how any meters (like reactor power or damage) can look — simple circular dials are intuitive and fit the tech theme.

The overall UI style: **clean lines, basic shapes, monochrome or monochrome-with-accent colors** (mirroring the facility’s color zones), and minimal ornaments. Text should use a monospaced or digital-looking font. When the temporal meter glitches, the HUD overlays should flicker or corrupt (random glyphs replacing text) to signal stability loss.

Art & Visual Style

RETRACE’s art will be **geometric and abstract**, avoiding detailed textures. Shapes (rectangles, triangles, circles) dominate — think *Thomas Was Alone* or minimalist sci-fi concept art. As one design guide notes, in a “geometric art style, the visuals revolve around basic shapes” 【61†L194-L201】. We lean into this: walls and floors are clean polygons, equipment is blocky, and even characters (player/MIMIC) have stylized forms (e.g. player = two blue glowing eyes on a cyan triangle cloak).

Lighting is crucial. The facility’s ambient color is **dark navy**; accent lights are region-dependent (e.g. cyan lights in Research, red in Security 【21†L0-L0】). Light pools are circular or hexagonal on the floor. Dynamic shadows from walls/corners enhance depth.

Sample Visual Inspirations:

【21†embed_image】 *Figure:* A neon-lit city with geometric architecture. Note the bold colored lights (cyan and magenta) and dark voids — similar contrast can highlight our corridors.

【14†embed_image】 *Figure:* A sci-fi turret with a single red sensor. Its angular design and exposed mechanics inspire our MIMIC or security robots.

【37†embed_image】 *Figure:* A close-up of an eye with digital glints. This can inform the design of MIMIC’s one large camera-eye (reflections, aperture).

【38†embed_image】 *Figure:* A real robot with a “camera” eye on its head. Use this as a reference for MIMIC’s silhouette – a sturdy base with a moving head-camera.

【39†embed_image】 *Figure:* A small rover-robot. Notice the simple geometric shapes and lights-on-its-body. Stylize MIMIC similarly (basic legs/wheels, big lens).

【40†embed_image】 *Figure:* Colorful cartoon robots. Even though RETRACE is serious, this shows how minimal shapes (circles, capsules) and expressive eyes give “personality” — we can apply a toned-down version to MIMIC (e.g. glowing eyelike sensor).

Walls and panels can have etched lines or patterns (think circuits or hex grids). We’ll use a *reusable room-decoration system*: wall-mounted pipes, vents, warning stripes, floor seams, and crates/tubes can be placed by code to add detail. This way each tilemap looks rich without handcrafting every room. Ambient particles (sparks near vents, dust) and flickering lights will add life.

(Citation on style: A geometric art style is intentionally simple but requires strong design. “Thomas Was Alone nailed this approach by turning rectangles into emotional... devices” [61†L194-L201] . RETRACE similarly uses basic forms to convey a high-tech facility atmosphere.)

Code Architecture & Data

The code is organized into systems and data models:

- **Level Format:** Use a simple tilemap format (e.g. CSV or JSON) for each room. Tiles: floor, wall, door, glass, etc. Entities (switches, spawners, patrol points) can be placed via an object layer in JSON. Naming convention: `level_<name>.json`.
- **Entities:**
 - *Player*: stores position, velocity, and action inputs.
 - *ECHO Clone*: each has a list of recorded “KeyFrames” (time, position, booleans for actions like pressing, interact).
 - *MIMIC*: state (Idle/Alert/etc), position, and **Memory**: counters for actions/locations. E.g. a dictionary mapping “nodeID”→visit count.
 - *Triggers*: Switches, Doors, Plates each have scripts that can be activated by either Player or Echo.
- **Event System:** A message bus handles global events (e.g. `OnDoorOpened`, `OnFootstep`) so that sound and MIMIC can subscribe.
- **Audio Manager:** Directional sound events produce both sound (via an audio source) and 2D UI cues (arrows).
- **Data Files:**
 - **Actions/Animation:** e.g. `player_spritesheet.png`, `mimic_spritesheet.png` (PNG, 64×64 or 128×128).
 - **Levels:** `.json` tilemaps.
 - **Echo Recording:** in-memory lists (no file, serialized only on save-state).
 - **Naming:** Follow snake_case for assets (e.g. `sprite_player.png`, `level_security.json`), CamelCase for classes.
 - **Determinism:** All ECHO playback is deterministic. No randomization in agent movement or physics. If needed, seed the RNG consistently (but best to avoid randomness entirely).

Data Models (conceptual):

```
// Example level JSON structure
{
  "name": "power_plant",
  "tiles": [[0,0,1,...], ...],
  "objects": [
    {"type":"switch", "x":10, "y":3, "id":"SW1"},
    {"type":"door",   "x":5,  "y":0, "id":"D1", "initial_state":"locked"}
  ]
}
```

- **EchoRecord:** [{t, x, y, pressAction, ...}, ...].
- **MIMICMemory:** { "used_run_speed": true, "visits_corridor": 3, ... }.

Networking / Save States

RETRACE is single-player, but serialization ensures consistency:

- **Save/Load:** On each RETRACE trigger or checkpoint, serialize the game state: tile states, player health/stability, which puzzles are solved, and **up to 3 Echo recordings** (binary or JSON). This allows perfect restoration on continue.
- **Echo Determinism:** Recorded ECHO data is stored frame-by-frame. On playback, ECHOs must move to identical positions. To guarantee this, locks on using physics interpolation (or snap to grid movement).
- **No external networking** is needed. But if future plans include multiplayer (racing two players through time), this structure can be extended.

Performance Considerations

- **Frame Rate:** Aim for 60 FPS on target hardware (modern PC). All core gameplay (movement, LOS, AI) should update each frame.
- **LOS Optimization:** Limit raycasts to a max radius. Use tile-grids to early-exit (cast to tile edges rather than per-pixel). Alternatively, maintain a shadow-casting algorithm on a coarse grid for performance.
- **Echo Count:** At most 3 Echos + player + MIMIC. If each record ~10 s run at 60Hz, that's ~600 frames * 3 = 1800 sample positions in memory (small).
- **AI Updates:** MIMIC's AI logic (line-of-sight checks) runs each frame but can be simplified: e.g. check LOS only when MIMIC is oriented to move.
- **Audio:** Limit simultaneous footstep sounds to avoid clipping. Usually one sound per moving agent per short interval.
- **Memory:** Preload sprite atlases and reuse particle systems. Avoid allocating objects each frame (use object pooling for bullets/sounds).
- **Profiling:** Key hotspots are LOS raycasts and physics. Use built-in profilers to ensure stable timing.

Testing & Acceptance Criteria

Each feature needs concrete tests:

- **Echo System:**
 - *Test:* Record a path, RETRACE, and verify an Echo follows the same path precisely.
 - *Acceptance:* Echo collision checks identify exactly same triggers hit as original. (Unit test: compare arrays of positions/actions.)
- **MIMIC AI:**
 - *Test 1:* Check that if player walks into MIMIC's cone of view, MIMIC transitions to Chase state immediately.
 - *Test 2:* If player makes noise out of sight, MIMIC goes to Alert and investigates.

- *Test 3:* After multiple runs, verify MIMIC's memory counters increment (e.g. using the same corridor increases its future visits).
- **LOS & Fog-of-War:**
- *Test:* In a simple map, verify areas behind walls are not rendered. Test that closed vs open doors change visibility.
- *Acceptance:* The visible region matches expected polygon from manual calculation (compare visible tile sets).
- **Sound Cues:**
- *Test:* Player steps at one location; an arrow appears in correct direction. If MIMIC is hearing, it turns toward sound.
- *Acceptance:* Direction indicator points correctly (e.g. up arrow if player north).
- **Temporal Stability:**
- *Test:* On first catch: confirm minor UI glitch (e.g. flicker); on second: more distortion; on third: full blackout and reset.
- *Acceptance:* Visual changes are progressive (check pixels change as expected), and after 0/3 life player is at checkpoint with counters reset.
- **Puzzle Solutions:** For each key puzzle:
- *Test:* Follow the intended Echo sequence (as per examples above) and verify the final objective unlocks.
- *Acceptance:* Door/puzzle only solves when correct ECHO actions are taken concurrently.

Checklists (each level/test scenario should have a pass/fail list). Use automated tests where possible (e.g. simulate inputs).

Prioritized Build Order

1. **Engine Setup & Player Controls:** Create a simple 2D project (no specific engine required, but e.g. Unity, Godot, or Pygame). Implement tilemap rendering and WASD movement.
2. **Line-of-Sight:** Implement LOS per above (simple raycasts/shadowcasting). Test with a static map (one room).
3. **Basic Level Layout:** Load level JSON or tilemap. Design a small test map with walls and doors.
4. **ECHO Recording/Playback:** Code recording of player transform/actions. Allow pressing R to restart. Verify that an Echo clone appears and moves correctly (no AI yet, just replay).
5. **Simple Puzzle (1-Echo):** Implement a pressure plate and door. Test with one Echo holding the plate.
6. **MIMIC Basics:** Create the MIMIC agent with patrol points. Add vision cone check and chase logic. Test chasing player.
7. **Sound Cues:** Add footstep sound events and directional indicator. Ensure MIMIC hears and reacts.
8. **Multi-ECHO Puzzles:** Using systems above, implement a 2-Echo puzzle (like two switches). Test.
9. **Additional ECHO Puzzles:** Add more puzzle types (cameras, lasers, etc). Iterate level design.
10. **UI Elements:** Create HUD overlays (ECHO count, interact prompt, hidden stability). Implement RETRACE meter UI.
11. **Temporal System:** Add life counter (hidden), glitch shaders or sprites, and reset logic after 3 catches.
12. **Polish Art/Animation:** Replace placeholders with geometric sprite art (see Prompt for creating art). Add animations (walk cycle, MIMIC eye movement, door slides).
13. **Lighting & Effects:** Add colored lights, shadow effects, screen shake when caught, glitch shaders.
14. **Audio:** Finalize footstep, door, ambience, alarm sounds.

15. **Testing & Balancing:** Playtest the loop. Adjust enemy speed, puzzle difficulty, echo limits.

If time runs short, **cut features** in this order: (1) Complex lighting/particles; (2) Polished animations (keep placeholders if necessary); (3) Optional puzzles or story logs; (4) Sound effects detail (basic beeps suffice).

Implementation Task Checklist & Claude Code Prompts

Below are key tasks broken into Claude Code prompts. Each prompt should be answered by Claude Code with specific code/assets.

- **Asset Creation (Sprites/Animations):**

- *Prompt:* "Create a 2D player character sprite sheet (PNG) in geometric style: a simple cloaked figure with two blue glowing eyes. Size 64×64, with idle and walk (4 frames) animations. Also create a MIMIC sprite sheet: a cube-on-wheels with one big red eye; include idle and rolling animations. Use solid color blocks and minimal shading."

- **Input:** Style notes (geometric, color palette).
- **Output:** Image files `player.png`, `mimic.png`, with defined frame regions.
- **Acceptance:** Sprites display correctly in engine, animations cycle smoothly.

- **Level Tilemap Loader:**

- *Prompt:* "Write code to load a tilemap from `level_power.json` (grid of integers). Convert integers to floor/wall sprites and place objects (door, switch) based on an `objects` list in the JSON. Use tags: 0=floor, 1=wall. Doors and switches have unique IDs."

- **Input:** Example `level_power.json` format.
- **Output:** A 2D level in the scene with correct sprites and colliders.
- **Acceptance:** The level file renders with floor and wall layout; doors and switches appear as separate objects at correct coordinates.

- **Player Controller:**

- *Prompt:* "Implement a top-down player controller (C# or GDScript): WASD moves the character sprite, Space = interact (talk/open), R = hold to retrace. Include basic collision with walls. Attach an 'eyes' sprite to indicate player direction."

- **Output:** A `PlayerController` script/component.
- **Acceptance:** Player moves smoothly, collides with walls, R triggers RETRACE (see next prompt).

- **Echo Recording System:**

- *Prompt:* "Create an Echo recording system: When the player holds R for >1s, save all recent actions as a new Echo and reset player to level start. Maintain up to 3 Echos (delete oldest beyond 3). On each frame, update each existing Echo's position/actions based on its recorded timeline. Ensure Echos' colliders/movement do not interact with player or each other."

- **Output:** Code classes `EchoManager`, `EchoAgent`.
- **Acceptance:** Pressing R records and spawns an Echo that follows the original path exactly; only 3 Echos ever exist.

- **Line-of-Sight (FOV):**

- *Prompt:* "Implement line-of-sight: Cast rays every 5° (or to corner of each tile) from player to range=6 tiles. Mark each hit tile as visible. Do not draw beyond walls/closed doors. Maintain an array of visible tiles for rendering."

- **Output:** `VisionSystem` script returning visible tile set each frame.

- **Acceptance:** Tiles blocked by walls are dark; open areas are illuminated. Tested on a sample map.
- **MIMIC AI Behavior:**
 - *Prompt:* "Implement the MIMIC enemy: uses a state machine (Patrol, Alert, Chase, Investigate). Patrol follows given waypoints. In Alert state, move toward sound source. In Chase, directly pursue player/Echo. If contact player, trigger RETRACE. Detect player/Echo via raycast LOS. If lost, go to Investigate for 3s, then return to Patrol."
 - **Output:** MimicAI component.
 - **Acceptance:** MIMIC moves along patrol; sees player and chases; loses target and returns to patrol.
- **Sound & Alerts:**
 - *Prompt:* "Add sound generation and UI cues: On each player/Echo footstep, play a footstep sound and, if the player is not visible, display an arrow at the screen edge pointing toward the sound. MIMIC subscribes to footstep events: if within 8 tiles, change state to Alert at source location."
 - **Acceptance:** Footsteps produce sound and arrow; MIMIC hears and turns.
- **Puzzle Logic (Switches & Doors):**
 - *Prompt:* "Write a generic TriggerSwitch script: when any agent (player or Echo) stands on it, it emits an event. A linked Door listens and opens (or closes) accordingly. Doors slide open (animated)."
 - **Acceptance:** An agent on plate opens door.
- **UI & HUD:**
 - *Prompt:* "Construct UI: a small panel showing Echoes: [●○○] for count. Contextual action text (e.g. [E] Open Door near interactive objects). No health bar. Minimal aesthetic."
 - **Acceptance:** UI elements appear in-game with correct count.
- **Temporal Glitch Effects:**
 - *Prompt:* "After 1st capture, overlay a quick red-tint flash. After 2nd, add scanlines and RGB split. After 3rd, fade to full static noise (using the glitch image textures [44],[47])."
 - **Acceptance:** Effects ramp up each capture as described.
- **Final Escape Logic:**
 - *Prompt:* "When power, security, and containment puzzles are solved, activate the final exit: open a large door with bright light effect. Upon reaching exit, display ending text."
 - **Acceptance:** Exit only opens after all objectives; ending triggers correctly.

Each prompt's **expected input** (design spec) and **output** (code or asset) are detailed above. For tests, ensure all features integrate seamlessly. If any detail (engine choice, specific sound files, etc.) is unspecified, assume a neutral setup (e.g. use Unity's default audio or Pygame, all pixel art 24-bit PNG).

Sources: We drew on examples of clone-based puzzles [50†L32-L40] , AI that learns player actions [58†L221-L229] , and standard LOS algorithms [53†L139-L148] , as well as art-style guidance [61†L194-L201] to inform our design. Each cited reference illuminates a key mechanic (time-clones, adaptive AI, 2D visibility, geometric graphics) that shapes RETRACE's design.
